

Student Version – Git & GitHub Case Study (Questions Only)

Scenario

You have joined XYZ Solutions as a Graduate Associate. Your team has been assigned to develop an **Online Bookstore**. The project already has a basic structure, and your responsibility is to use Git and GitHub to manage the source code and collaborate with your teammates. Complete the following tasks. Do not ask your instructor for the commands—identify and use the appropriate Git commands yourself.

Answer all questions by performing the tasks on your system.

1. Create a new project folder named **OnlineBookStore** and initialize it as a Git repository.
2. Create the following files and folders: README.md, index.html, css/style.css, js/app.js, books/books.txt. What does Git report after you create them?
3. Stage only README.md and index.html. Which command did you use? How can you verify only these files are staged?
4. Stage the remaining files and create your first commit with an appropriate commit message.
5. Modify README.md by adding project objectives. Without committing, display the changes made to the file.
6. Create a branch named **feature-catalog** and switch to it. How do you verify the active branch?
7. Update books/books.txt with five sample book titles and commit the changes.
8. Switch back to the main branch. Has your new file content appeared there? Explain your observation.
9. Merge the feature-catalog branch into the main branch and verify the merge history.
10. Create a GitHub repository named **OnlineBookStore** and connect your local repository to it.
11. Push your project to GitHub. Verify that all files are available in the remote repository.
12. Create another branch named **feature-search**. Add search.html and commit your changes.
13. Push the feature-search branch to GitHub and create a Pull Request. Record the title you used for the PR.
14. Your teammate updated the main branch. Retrieve the latest changes into your local repository.
15. Accidentally create a file named notes.txt and stage it. Remove it from the staging area without deleting the file.
16. Display the complete commit history in graph format. Identify the merge commit.
17. Explain the difference between a branch merge and a Pull Request.
18. List three situations where Git helps software development teams.
19. Describe one situation that can lead to a merge conflict and how you would resolve it.
20. Submit your GitHub repository URL, screenshots of commit history, branch list, Pull Request, and final repository.

Submission Checklist

- GitHub Repository URL
- Screenshot of git status
- Screenshot of commit history
- Screenshot of branches
- Screenshot of Pull Request

- Screenshot of merged repository
- Short reflection (5–10 lines): What did you learn from this activity?